National University of Computer & Emerging Sciences

Lecture 8

Process Creation

Process Creation and Termination

Creation	Termination
New batch job	Normal Completion
Interactive Login	Memory unavailable
Created by OS to provide a service	Protection error
Spawned by existing process	Operator or OS Intervention

What's Happening at Process Creation?

- The OS:
  - Assigns a unique process identifier
  - Allocates space for the process
  - Initializes process control block
  - Creates or expand other data structures
- Traditionally, the OS created all processes
  - But it can be useful to let a running process create another
  - This action is called ***process spawning***
    - ***Parent Process*** is the original, creating, process
    - ***Child Process*** is the new process

OS Control Tables

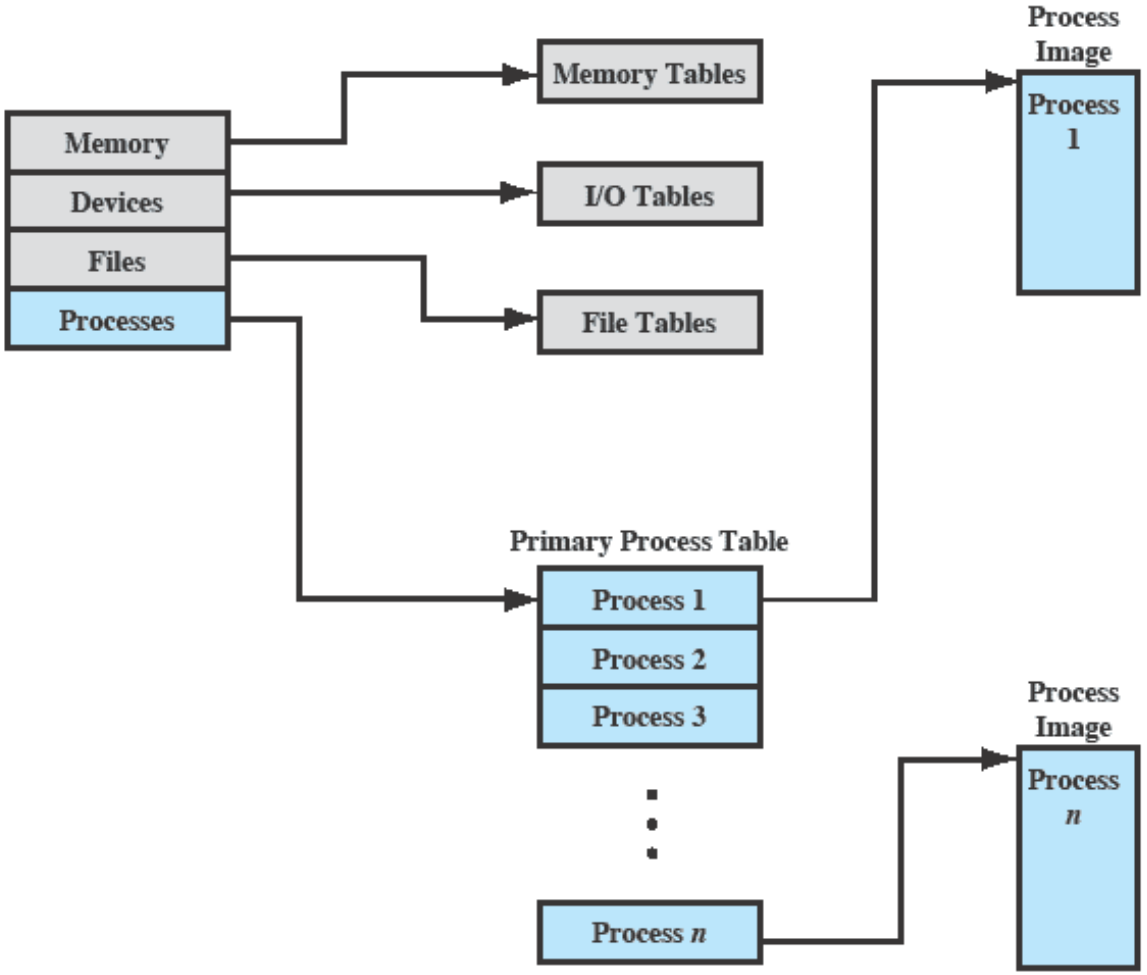


Figure 3.11 General Structure of Operating System Control Tables

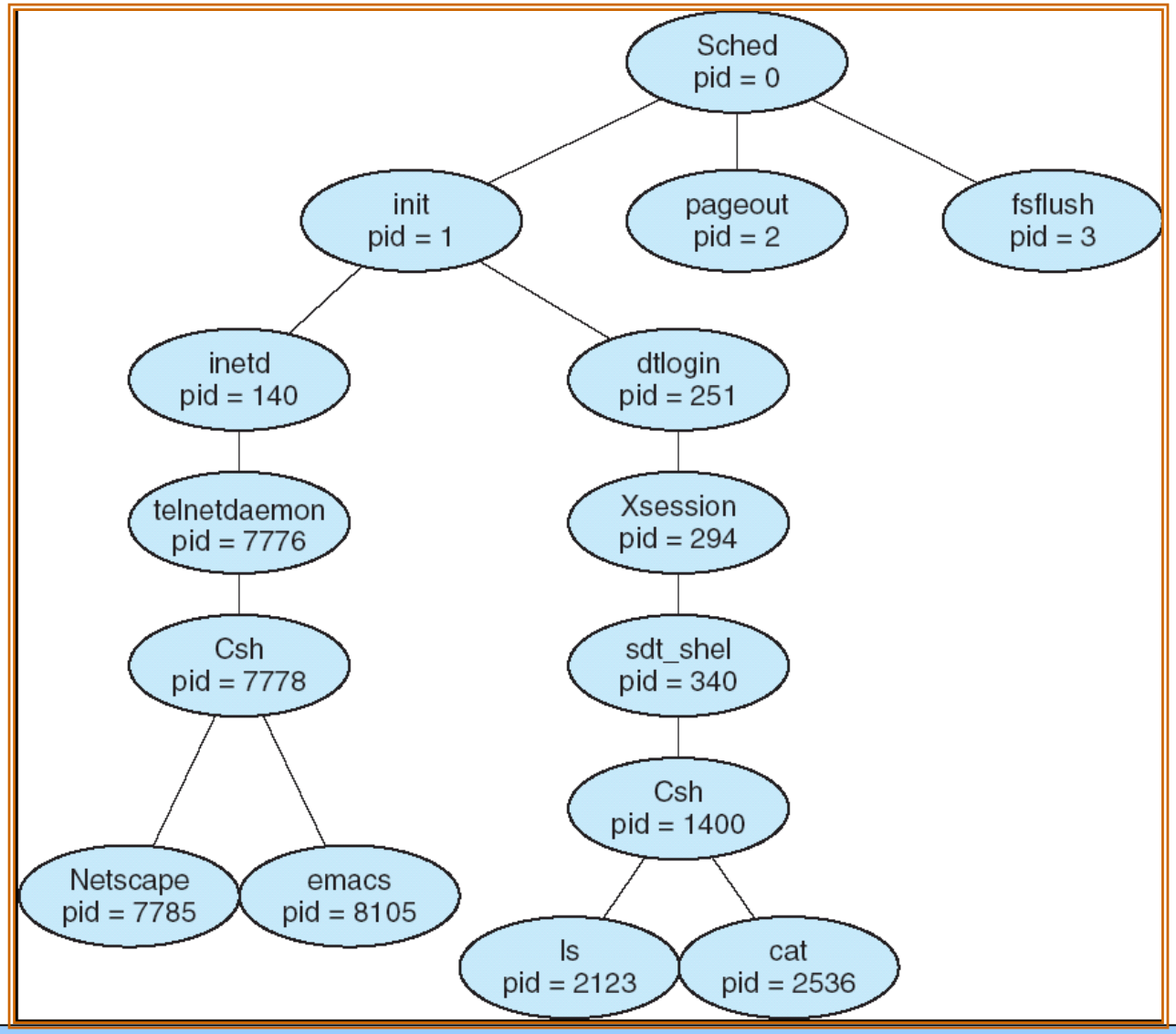
Process Termination

- Batch job issues Halt instruction
- User logs off
- Process executes a service request to terminate
- Parent terminates so child processes terminate
- Operating system intervention
  - such as when deadlock occurs
- Error and fault conditions
  - E.g. memory unavailable, protection error, arithmetic error, I/O failure, invalid instruction

Unix Process Creation

- When the system starts up it is running in kernel mode
- There is only one process, the initial process.
  - **SCHED process with PID 0**
- At the end of system initialization, the initial process starts up another kernel process.
  - The **init** kernel process has a process identifier of 1.

A Tree of Processes



Process Creation

- These new processes may themselves go on to create new processes.
- All of the processes in the system are descended from the init kernel thread.
- You can see the family relationship between the running processes in a Linux system using the **pstree** command
- A new process is created by a **fork()** system call

Compiling C++ code

- g++ test.cpp –o Output
- Running the code:
- ./Output

The fork() system call

At the end of the system call there is a new process waiting to run once the scheduler chooses it

- A new data structure is allocated
- The new process is called the child process.
- The existing process is called the parent process.
- The parent gets the child's pid returned to it.
- The child gets 0 returned to it.
- Both parent and child execute at the same point after fork() returns

Unix Process Control

```
int main()
{
        int pid;
        int x = 0;

        x = x + 1;
        fork();
        x = 3;
        printf("%d",x);
}
```

But we want the child process to do something else...

```
int pid;
int status = 0;

pid = fork();
if (pid>0) {
      /* parent */
      ……
      pid = wait(&status);
} else {
      /* child */
      ……
      exit(status);
}
```

*The **fork** syscall returns a zero to the child and the child process ID to the parent*

*Parent uses **wait** to slee… chil… retu… and…*

***Fork** creates an exact copy of the parent process*

***Wait** variants allow wait on a specific child or*

*Child process passes status back to parent on **exit**, to report success/failure*

Child Process Inherits

- Stack
- Memory
- Environment
- Open file descriptors.
- Current working directory
- Resource limits
- Root directory

Child process DOESNOT Inherit

- Process ID
- Different parent process ID
- Process times
- Own copy of files
- Resource utilization (initialized to zero)

How can a parent and child process communicate?

- Through any of the normal IPC mechanism schemes.
- But have special ways to communicate
  - For example
    - The variables are replicas
    - The parent receives the exit status of the child

The wait() System Call

- A child program returns a value to the parent, so the parent must arrange to receive that value
- The wait() system call serves this purpose
  - pid_t wait(int *status)
  - it puts the parent to sleep waiting for a child's result
  - when a child calls exit(), the OS unblocks the parent and returns the value passed by exit() as a result of the wait call (along with the pid of the child)
  - if there are no children alive, wait() returns immediately
  - also, if there are zombies, wait() returns one of the values immediately (and deallocates the zombie)

What is a zombie?

- In the interval between the child terminating and the parent calling wait(), the child is said to be a ‘zombie’.
- Even though its not running its taking up an entry in the process table.
- The process table has a limited number of entries.

What is a zombie?

- If the parent terminates without calling wait(), the child is adopted by init.

The solution is:

- Ensure that your parent process calls wait() or waitpid or etc, for every child process that terminates.